

目录

1. 项目简介	3
1.1 项目整体介绍	3
1.2 项目背景信息	3
1.3 项目要求	3
2. 可行性分析	4
2.1 技术可行性	4
2.2 时间可行性	4
3. 项目需求	5
3.1 功能性需求	5
3.2 非功能性需求	6
4. 项目功能设计及实现	6
4.1 总体设计	6
4.2 详细设计及实现	7
4.2.1 注册与登录模块	7
4.2.2 遍历算法模块	9
4.2.3 主界面展示模块	12
4.2.4 绘制模块	16
4.2.5 界面美化模块	17
5. 总结	18
5.1 项目总结	18
5.2 心得体会	19
5.3 致谢	20

1. 项目简介

1.1 项目整体介绍

该项目是一个基于 C++ 和 Qt 框架开发的图形界面应用程序，旨在解决的“马踏棋盘”问题。在一个 8×8 的国际象棋棋盘上，任意选择起点位置并放置一匹马，使其能够恰好访问每个方格一次并且返回起点。

本项目完成了一个软件，让用户可以通过图形界面选择马的起始位置，并观察马如何遍历整个棋盘并返回起点。

1.2 项目背景信息

国际象棋的棋盘有 $8 \times 8 = 64$ 个格子。马在这 64 个格子的某一个格子上，它的跳动规则是：如果它现在在 (x, y) 位置，它下一步可以跳到 $(x \pm 1, y \pm 2)$ 或 $(x \pm 2, y \pm 1)$ （所有的“ $\pm$ ”之间没有相关性），每个位置下一步最多可以有八种跳法。但是不能跳出这 64 个格子。

1.3 项目要求

- (1) 设计算法使马不管从哪出发都可以跳遍所有的格子（每个格子只能路过一次）
- (2) 最后回到起点。
- (3) 对算法进行优化，根据 j. c. Warnsdorff 规则设计算法，该规则是在所有可跳的方格中，马只可能走这样一个方格：从该方格出发，

马能跳的方格数为最少；如果可跳的方格数相等,则从当前位置看,方格序号小的优先。

2. 可行性分析

2.1 技术可行性

(1) 开发语言与框架支持

项目采用 C++ 和 Qt 框架进行开发,C++ 作为一种高效且应用广泛的编程语言,具备对复杂算法实现的支持。Qt 框架则在图形界面开发方面有优势,能够方便地创建交互式且美观的图形界面应用程序,满足软件现注册登录界面、棋盘展示、动画展示等功能,从技术层面上为项目实现提供了可靠的基础。

(2) 算法实现可行性。

通过递归、回溯以及剪枝操作,结合 j. c. Warnsdorff 规则能够较快运行出遍历顺序结果,理论上算法可行。

2.2 时间可行性

虽然软件涉及多个功能模块,但各功能模块相对来说逻辑较为清晰。在合理规划下,先完成核心的“棋盘遍历”算法模块,再逐步完善界面相关功能等。在既定的实训课程期限内完成项目开发是可行。

3. 项目需求

3.1 功能性需求

(1) 设计注册与登录界面，提供用户登录和注册功能。

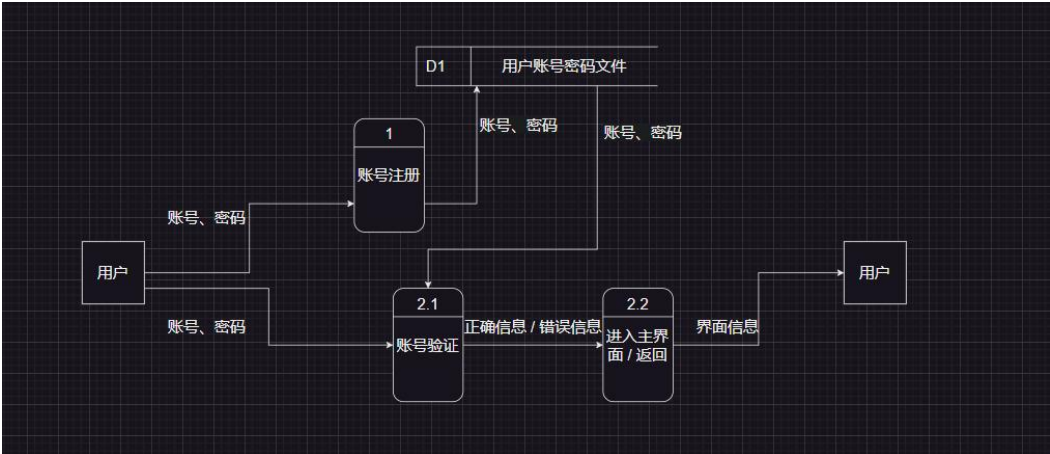


图 3-1 用户登录与注册功能数据流图

(2) 根据 j. c. Warnsdorff 规则优化实现马的移动逻辑，确保马在任意位置开始能够访问棋盘上每一个格子恰好一次并且返回起点。

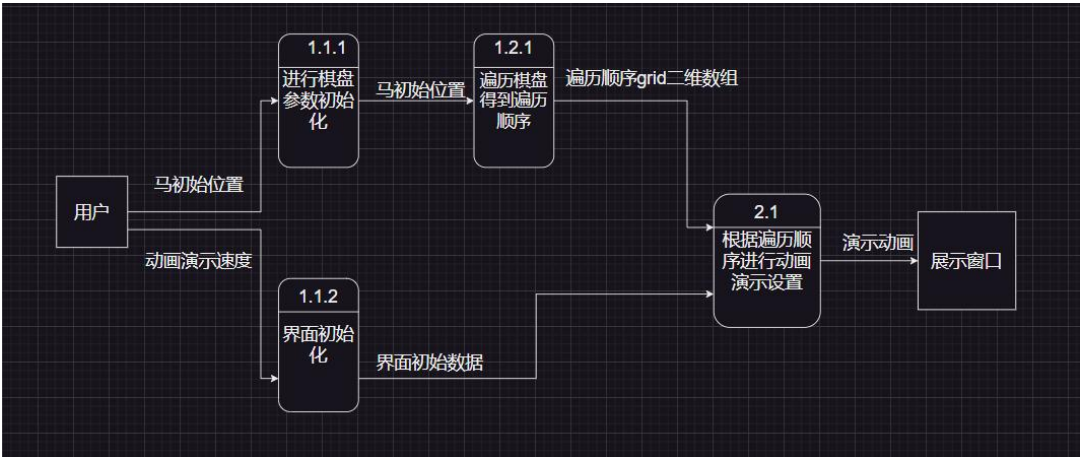


图 3-2 棋盘遍历与展示功能数据流图

(3) 提供一个棋盘，用户能够选择马的起始位置。

(4) 用户可以选择不同的遍历展示速度。

(5) 提供重置功能，使得用户能够重新选择起点或遍历展示速度进行遍历。

(6) 实现马踏棋盘的动画展示。

3.2 非功能性需求

- (1) 界面友好美观，易于用户进行操作。
- (2) 程序响应迅速，动画流畅。
- (3) 代码可读性好，易于维护和扩展。

4. 项目功能设计及实现

4.1 总体设计

本软件主要分为五个模块，分别是注册登录模块、遍历算法实现模块、主界面展示模块、绘制模块、界面美化模块。

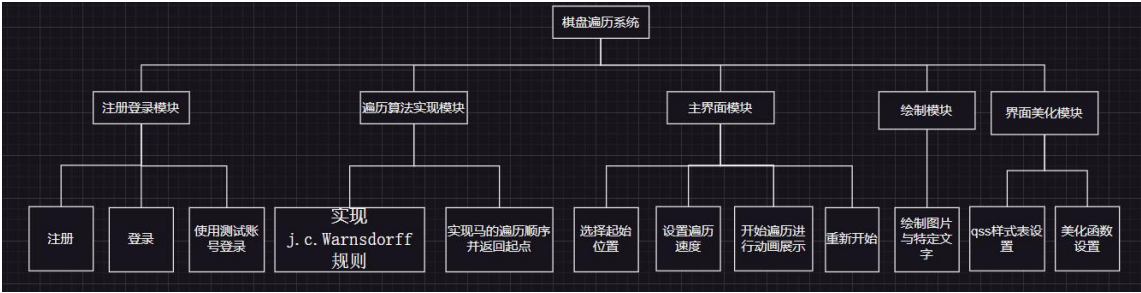


图 4-1 软件总体设计层次图

- (1) 注册登录模块实现用户的注册、登录和测试账号登录功能。通过账号与密码，软件会对用户身份进行基本的校验。
- (2) 遍历算法模块首先实现 j. c. Warnsdorff 规则，然后实现遍历过程中调用根据 j. c. Warnsdorff 规则实现的排序函数，进行遍历，最终得到遍历顺序与二维数组。
- (3) 在主展示界面模块中设置棋盘，实现马在棋盘中的动画遍历展示，并实现重新开始功能。同时打造友好且美观的界面。

(4) 绘制模块完成棋盘中马的图像绘制以及棋盘遍历完成后的数字顺序图形绘制。

4.2 详细设计及实现

4.2.1 注册与登录模块

(1) 注册功能

实现与注册按钮相连接的槽函数

`on_register_2_clicked`，让用户输入要注册的账号与密码，判断账号或密码是否合法，若不合法，则返回“请重新输入”的警告，若输入账号和密码合法，则执行 `registerUser` 函数，将用户输入的账号、密码作为参数传入该函数。

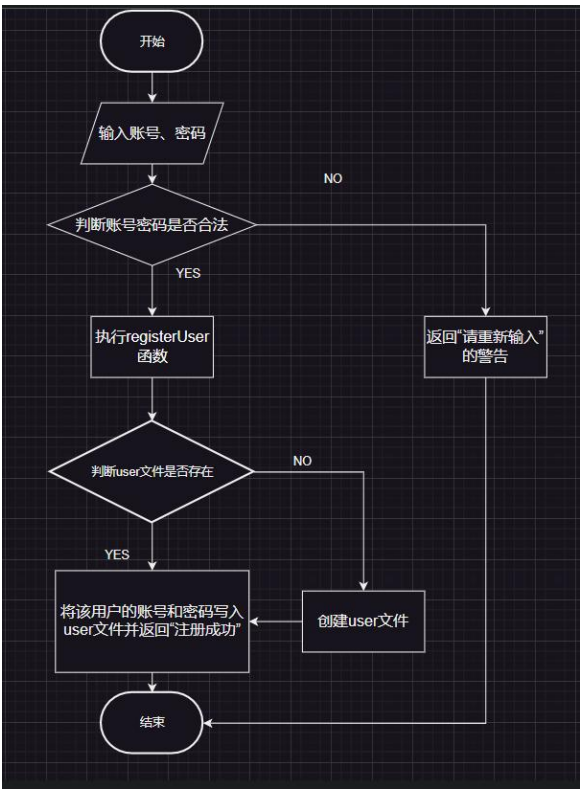


图 4-2-1 注册功能程序流程

在 `registerUser` 函数中，首先判断 `user` 文件是否存在（该文件用来存放用户的账号和密码）。若文件存在，则将该用户的账号与密码存入文件，若文件不存在，先创建文件，然后将该用户的账号和密码存入文件。完成用户注册。

```
QString filePath = QApplication::applicationDirPath() + "/user.txt";
QFile file(filePath);
```

```

if (!file.open(QIODevice::Append | QIODevice::Text)) {
    return false;
}
QTextStream out(&file);
out << username << "," << password << "\n";
return true;

```

（2）登录与使用测试账号登录功能

当用户点击登录按钮时，`on_login_2_clicked` 槽函数被触发，首先从账号输入框与密码输入框中获取用户输入的账号与密码。调用 `check_login` 函数并传入获取到的账号和密码进行登录验证。

在 `check_login` 函数中，首先判断账号和密码是否为测试账号和测试账号对应的密码，若是，怎之间返回 `true`，否则遍历 `user` 文件中所用用户数据，若该账号与密码在 `user` 文件中存在且对应，则返回 `true`，否则返回 `false`。

根据 `check_login` 函数的返回值判断，若为 `true`，则登录成功，进入主界面，若为 `false`，则返回“登录失败”的警告，让用户进行重新登录。

```

bool login::check_login(const QString &username, const QString &password)
{
    // 使用测试账号登录验证

```

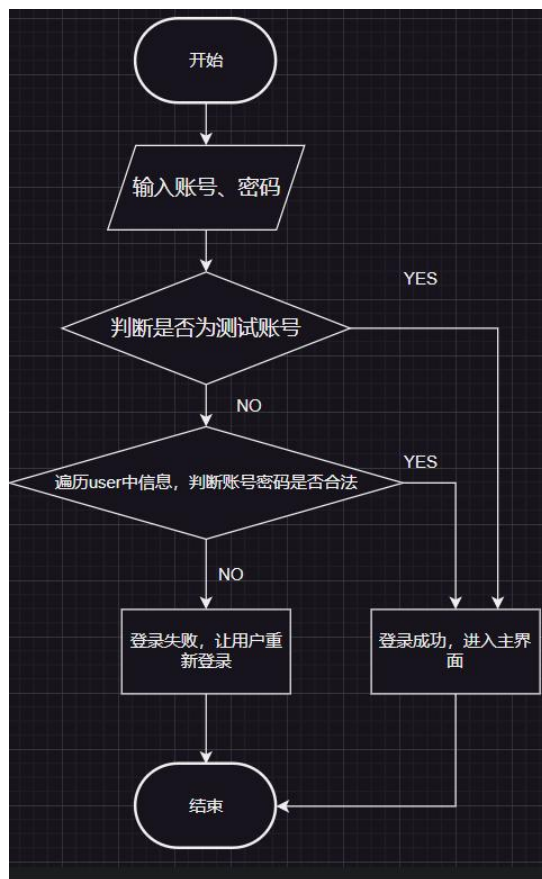


图 4-2-2 登录功能程序流程图

```

        if(username=="2307020323"&&password=="123456"){
            return true;
        }
        //其他登录验证
        QString filePath = QApplication::applicationDirPath() +
"/user.txt";
        QFile file(filePath);
        if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
            return false;
        }
        QTextStream in(&file);
        QString line;
        while (!in.atEnd()) {
            line = in.readLine();
            QStringList fields = line.split(",");
            if (fields.size() == 2 && fields[0] == username && fields[1]
== password) {
                file.close();
                return true;
            }
        }
        file.close();
        return false;
    }
}

```

4.2.2 遍历算法模块

(1) 实现 j. c. Warnsdorff 规则

在每次走下一步之前，计算 8 个可能位置的下一步可行步数，优先选择后续可走位置较少的方向，以更高效地遍历整个棋盘。

在最初，初始化 `dir_x[8]`、`dir_y[8]` 的数组，每一个索引 `i` 代表一个方向，并初始化 `j_c_Warnsdorff_sortarray[8]` 数组，代表 8 个方向的优先级顺序。在走下一步前调用 `sort_j_c_Warnsdorff` 函数。

然后首先初始化数组 `sorted_by[8]`，代表 8 个方向的后续可行步数。然后通过两次循环，得到每一个方向后续可走位置的数量，存入 `sorted_by` 数组中，然后调用 `sort_index` 函数，根据 `sorted_by` 数组中存储的每一个位置的可行步数对 `j_c_Warnsdorff_sortarray[8]` 数组进行排序，得到后续位置遍历的优先级顺序，从而实现 `j_c_Warnsdorff` 规则。

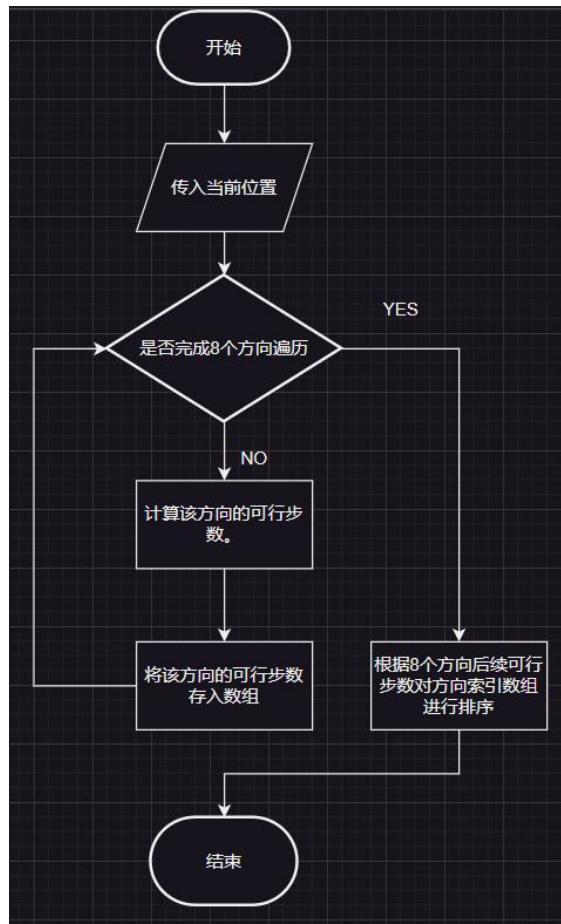


图 4-2-3 j. c. Warnsdorff 排序程序流程图

```

void horse::sort_j_c_Warnsdorff(int x, int y)
{
    int next_x1, next_y1, next_x1_next, next_y1_next;
    int sorted_by[8] = {8, 8, 8, 8, 8, 8, 8, 8};
    for (int i = 0; i < size_x; i++)
    {
        int step_cnt = 0;
        next_x1 = x + dir_x[i], next_y1 = y + dir_y[i];
        if (in_grid(next_x1, next_y1) == 1)
        {
            for (int j = 0; j < size_y; j++)
            {
                next_x1_next = next_x1 + dir_x[j], next_y1_next = next_y1
+ dir_y[j];
                if (in_grid(next_x1_next, next_y1_next) == 1) {
                    step_cnt++;
                }
            }
        }
    }
}
  
```

```

    }
    sorted_by[i] = step_cnt;
}
sort_index(sorted_by, j_c_Warnsdorff_sortarray, size_x); //排序
}

```

(2) 实现马遍历棋盘并返回起点的算法

首先对一些数据进行初始化操作，避免上次遍历结果影响本次遍历。

调用 `hores_traversal` 函数进行遍历，得到代表遍历顺序的二维数组。在这个过程中，首先对于几个状态较多位置进行贪心处理，减少程序运行时间，做到程序迅速响应。然后进行遍历操作。

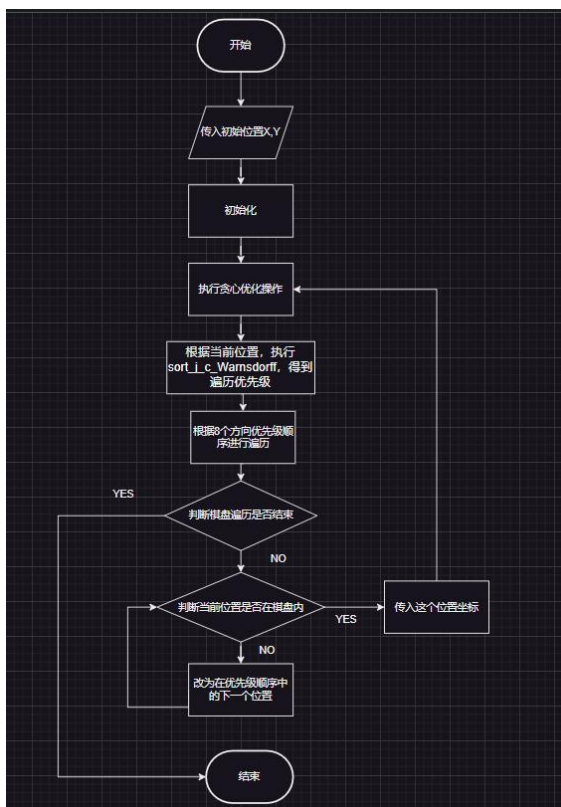


图 4-2-4 棋盘遍历程序流程图

在每次运行中调用 `sort_j_c_Warnsdorff` 函数，得到后续位置的优先级排序，根据优先级排序进行走下一步，计算下一个位置的 X、Y 坐标，每次先判断已经遍历的位置数，当达到 64 时，且最后一个位置能够回到起点，递归程序结束。然后调用 `in_grid` 函数判断下一个位置是否在棋盘内，若是，则进行下一次递归，若这次递归返回值为 1，则继续，若返回 0，则代表当前位置不符合条件，并将当前棋盘位置还原为初始值。

最终已经遍历的位置数达到 64 时，且最后一个位置能够回到起点，递归程序结束，完成棋盘遍历操作。

```
int horse::horses_traversal(int x, int y, int deep)
{
    every_init(deep);
    int i, next_x1, next_y1;
    sort_j_c_Warnsdorff(x, y); // 对八个方向进行排序
    for (i = 0; i < able_step; i++)
    {
        next_x1 = x + dir_x[j_c_Warnsdorff_sortarray[i]];
        next_y1 = y + dir_y[j_c_Warnsdorff_sortarray[i]];
        int absx = abs(next_x1 - begin_x + 1);
        int absy = abs(next_y1 - begin_y + 1);
        if (in_grid(next_x1, next_y1) == 1) // 检查
        {
            grid[next_x1][next_y1] = deep;
            int go_to_begin=absx * absx + absy * absy;
            if (deep >= Total_step && (go_to_begin == 5)) {
                return 1;
            }
            if (horses_traversal(next_x1, next_y1, deep + 1) == 1) {
                return 1;
            }
            else {
                grid[next_x1][next_y1] = 0; // 将棋盘还原
            }
        }
    }
    return 0;
}
```

4.2.3 主界面展示模块

(1) 选择起始位置

在主界面 UI 中使用 8*8 表格设置棋盘
在每一个表格上设置一个按钮，若点

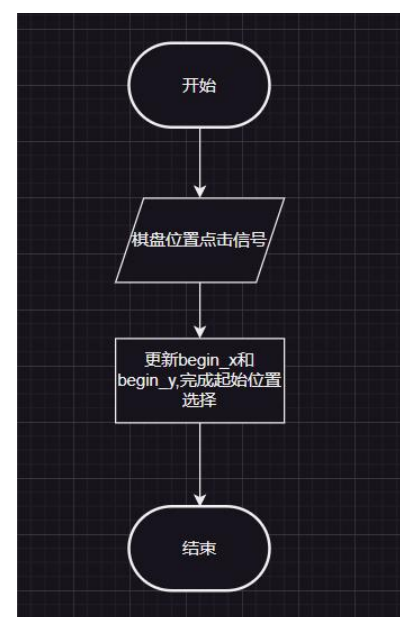


图 4-2-5 选择初始位置功能程序流程图

击则代表选择这个位置为初始位置，将位置参数传到遍历算法模块，更新初始位置 `begin_x` 和 `begin_y`。从而完成初始位置选择功能。

```
QPushButton *bu=new QPushButton("",this);
ui->grid->setCellWidget(i,j,bu);
connect(bu,&QPushButton::clicked, [=]() {onBuClicked(i,j);});

void MainWindow::onBuClicked(int row, int column) {
    horse::begin_x=row+1;
    horse::begin_y=column+1;
}
```

（2）设置遍历速度

使用 `init_sec` 函数将可用速度放入 `vector<pair<int,int>>` 容器中，然后设置一个全局变量 `sec_chose`，作为速度选择器，其数值与该 `vector` 向量的下标对应。

在 UI 界面中设置两个按钮“快速遍历”和“细致演示”，进行速度选择。将两个按钮分别与 `on_re1_clicked` 和 `on_re2_clicked` 槽函数连接，在点击时更新 `sec_chose` 的值，从而实现不同遍历速度的设置。

```
void MainWindow::init_sec() {
    sec.resize(2);
    sec[0]={300,250};
    sec[1]={1000,900};
}

void MainWindow::on_re2_clicked()
{
    sec_chose=1;
```

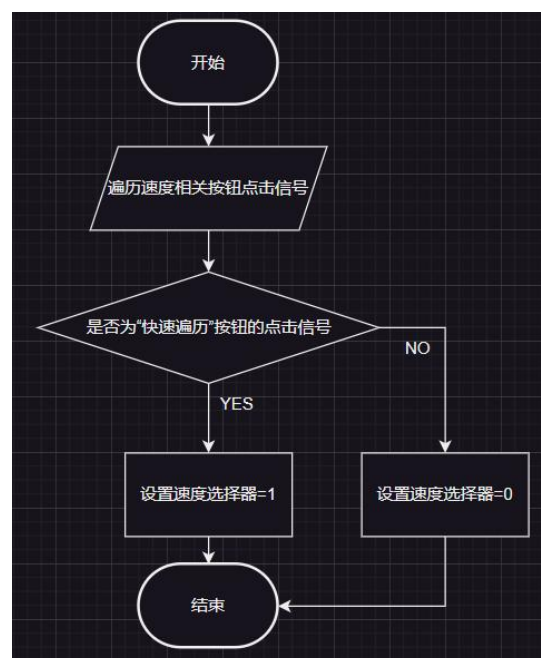


图 4-2-6 设置遍历速度功能程序流程图

```

}

void MainWindow::on_re1_clicked()
{
    sec_chose=0;
}

```

(3) 开始遍历与动画展示功能

在 UI 界面上设置“开始”按钮，将按钮与 `on_begin_clicked` 槽函数相连接。点击开始后，首先运行在棋盘遍历模块已经实现的 `do_hores_traversal` 函数得到棋盘遍历顺序，然后运行 `show_horse_traversal` 函数进行界面的动画展示，同时使用 `QTimer` 功能设置在程序运行时，部分功能性按钮不可选中，避免程序运行时错误。

在动画展示功能中，`show_horse_traversal` 函数为该功能的起始函数，在该函数内调用 `set_pic` 函数进行棋盘格子图片设置和经过后遍历标记设置，同时设置 `Timer`，超时后运行 `delayedFunction` 函数，达到延迟显示的效果，从而实现动画效果。

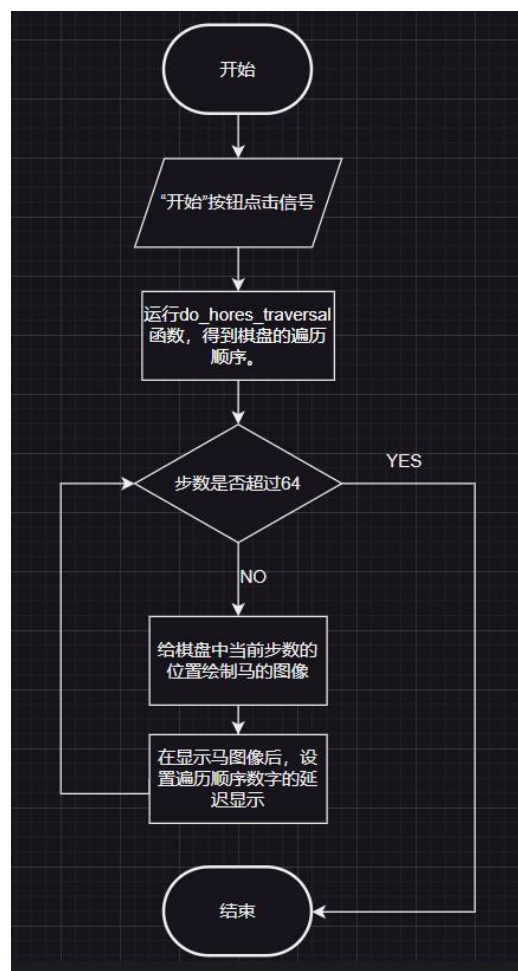


图 4-2-7 开始与动画展示功能程序流程图

```

void MainWindow::show_horse_traversal() {
    for (int i = 0; i<8; i++) {

```

```

        for (int j = 0; j<8; j++) {
            hash_grid[horse::grid[i][j]] = {i, j};
        }
    }
    for (int i = 0; i <= 64; i++) {
        if(i==64){
            i=65;
        }
        QTimer::singleShot(sec[sec_chose].first*i, this, [this, i]() {
            auto it = hash_grid.find((i + 1)%65);
            if (it != hash_grid.end()) {
                set_pic(it->second);
            }
        });
    }
}

connect(timer, &QTimer::timeout, this, &MainWindow::delayedFunction);

connect(this, &MainWindow::picSet, this, &MainWindow::delayedFunction,
Qt::QueuedConnection);

void MainWindow::set_pic(pii xy)
{
    change_xy = xy;
    QPixmap pixmap0(img_path[img_horse]);
    MyDelegate *delegate = new MyDelegate;
    ui->grid->setItemDelegate(delegate);
    QIcon icon(QIcon::fromTheme(img_path[img_horse]));
    QTableWidgetItem *item1 = new QTableWidgetItem();
    item1->setData(Qt::UserRole + 1, QVariant::fromValue(icon));
    ui->grid->setItem(xy.first, xy.second, item1);
    emit picSet(xy); // 发送信号
    timer->stop();
    timer->start(sec[sec_chose].second);
}

```

(4) 重新开始功能

将重新开始按钮连接到槽函数 `on_rebegin_clicked`，当点击重新开始按钮时，则执行这个函数。

在 `on_rebegin_clicked` 调用页面初始化函数 `init(-1, -1)`，同时保持 `change_xy` 为最初值，其中参数不在 0-7 之间，代表还未选择起始位置。然后调用 `set_green_theme` 设置页面主题，再停止 timer 计时，防止前一次遍历影响本次遍历的结果。

```
void MainWindow::on_rebegin_clicked()
{
    horse::init(-1, -1);
    change_xy = {-1, -1};
    set_green_theme();
    be_timer->stop();
}
```



图 4-2-8 重新开始功能程序流程图

4.2.4 绘制模块

定义 `MyDelegate` 类，使其继承 `QStyledItemDelegate`。然后重写父类的同名函数 `paint` 函数，绘制单元格背景和边框，然后获取马的图像和当前单元格的文本，根据单元格大小调整图标尺寸，并计算图标和文本的绘制位置，最后在相应位置绘制图标和文本，实现对表格项显示内容和样式的自定义，解决直接设置图片而导致图片显示位置偏左的问题。

```
void MyDelegate::paint(QPainter *painter, const QStyleOptionViewItem &option, const QModelIndex &index) const {
    QStyledItemDelegate::paint(painter, option, index);
    QIcon icon = qvariant_cast<QIcon>(index.data(Qt::UserRole + 1));
```



```

        QString text = index.model()->data(index,
Qt::DisplayRole).toString();
        QPixmap pixmap = icon.pixmap(option.rect.size());
        QRect iconRect(option.rect);
        QRect textRect(option.rect.adjusted(pixmap.width() + 4, 0, 0, 0));
        painter->drawPixmap(iconRect.topLeft(), pixmap);
        painter->drawText(textRect, Qt::AlignLeft | Qt::AlignVCenter,
text);
    }

```

4.2.5 界面美化模块

效果图展示：



图 4-2-9 注册登录界面图



图 4-2-10 主界面图

(1) qss 文件设置

在 `main` 函数中读取写好的 `qss` 文件，然后将 `qss` 文件以只读方式打开，再进行设置样式表，达到美化界面的效果

```

QFile qss(":/my_sh.qss");
qss.open(QFile::ReadOnly);
a.setStyleSheet(qss.readAll());

```

下面是 `my_sh.qss` 文件部分内容展示：

```

QPushButton#begin:hover { /* 鼠标悬停状态 */
    background-color: rgb(126, 255, 219);
    color: white;
}
QPushButton#begin:pressed { /* 按钮按下状态 */
    background-color: transparent;
}

```



```
        color: rgb(126, 255, 219);  
    }  
}
```

(2) 进行界面美化的函数设置

运用 Qt 中的一些函数进行界面美化,如 `setPalette`、`setBackground`、`setIcon` 等函数。

```
QPixmap pixl=QPixmap(":/image/1.jpg").scaled(this->size());  
QPalette palette;  
palette.setBrush(QPalette::Background,QBrush(pixl));  
this->setPalette(palette);  
  
ui->grid->item(i, j)->setBackground(QColor(222, 252, 249));  
  
this->ui->min->setIcon(QIcon(":/image/min.png"));
```

5.总结

5.1 项目总结

本实训项目实现了一个具有登录注册功能的“棋盘遍历”应用程序。

在登录模块,用户可输入账号密码进行登录验证,若账号密码正确则可进入主界面,错误时会有相应提示信息弹出。注册功能允许用户创建新账号并将其信息存储到文件中。

主界面展示了 8x8 的棋盘,用户能够通过点击棋盘格子选择马的起始位置,随后程序运用 j_c_Warnsdorff 规则和递归、回溯、贪心等算法对马在棋盘上的遍历路径进行计算,并以动画形式展示遍历过

程，将马的位置变化以及棋盘格的标记逐步呈现给用户，同时还具备界面设置、最小化、关闭以及调整遍历时间间隔等功能。

整个实训项目包括了图形界面设计、用户登录注册处理、算法实现以及文件操作等技术。

5.2 心得体会

通过本次实训课程作业，我收获颇丰。

在技术层面，在大一小学期学习的基础上，我更加深入学习了 Qt 框架的使用，熟练掌握了如何创建图形化用户界面，包括各种控件的使用（如 QPushButton、QTableWidget、QLineEdit 等）。通过此次实训课程，我更加深入学习了信号与槽机制，并熟练运用这个机制来实现用户交互逻辑。例如，在登录界面与主界面的切换、棋盘格子点击事件的处理等方面。

对于文件操作，我也有了更扎实的理解与应用能力（如登录验证时读取用户信息文件、注册时在文件中写入新用户信息）。

在算法方面，实现“马踏棋盘”的遍历算法让我对递归算法、回溯算法和贪心算法等算法有了直观的认识与理解。这不仅锻炼了我的逻辑思维能力，也让我体会到算法在实际应用场景中的重要性与复杂性。

在项目开发过程中，我也意识到良好的代码结构与规范的重要性。在进行项目开发过程中，我深刻地认识到写注释的重要性，我也体会到了《软件工程导论》中的一个观点——“文档与代码同等重要”。

同时也意识到合理的类设计与函数划分能够使代码的可读性与可扩展性大大提高。

然而，我在项目开发过程中也遇到了一些问题。例如，在实现动画展示遍历过程时，对于 QTimer 的信号传递的理解不够深入，导致出现一些显示异常，经过反复查阅资料与调试才得以解决。这让我更加深刻地明白在学习时，不能仅仅停留在表面的功能实现，更要深入理解其背后的原理与机制。

对我来说，本次实训课程是一次非常宝贵的学习经历，让我将所学的理论知识与实际开发相结合，让我更加明确了自己在编程技术上的优势与不足，为进一步提升自己指明了方向。

5.3 致谢

最后，感谢张老师和孙老师的悉心教导与帮助。在数据结构课程和实训课程中，老师们都给了我许多帮助与指导，让我能够学到了很多知识和技能。在数据结构课程中，张老师的讲解让我对数据组织和存储以及一些算法有了更深刻的理解，这对于我在实训课程中设计和优化算法很有帮助。

祝老师身体健康，工作顺利！